

BANCO DE DADOS RELACIONAL

Subconsultas (subqueries)

Professora:

Lucineide Pimenta

Recapitulando a aula anterior

✓ Na Aula 12 vimos **GROUP BY** com múltiplos JOINS e **HAVING**.



BANCO DE DADOS RELACIONAL

Sistema de Biblioteca - Modelagem e Implementação do Banco de Dados

Modelo Entidade-Relacionamento (MER)

- ❑ Entidades principais:
- ❑ **Autor**
 - ❑ id_autor (PK)
 - ❑ nome
- ❑ **Livro**
 - ❑ id_livro (PK)
 - ❑ titulo
 - ❑ ano_publicacao
 - ❑ id_autor (FK → Autor)
- ❑ **Aluno**
 - ❑ id_aluno (PK)
 - ❑ nome
 - ❑ curso
- ❑ **Emprestimo**
 - ❑ id_emprestimo (PK)
 - ❑ data_emprestimo
 - ❑ id_aluno (FK → Aluno)
- ❑ **EmprestimoLivro** (associativa N:M)
 - ❑ id_emprestimo (FK → Emprestimo)
 - ❑ id_livro (FK → Livro)
- ❑ **Assim temos:**
 - ❑ Relação 1:N entre Autor → Livro.
 - ❑ Relação 1:N entre Aluno → Emprestimo.
 - ❑ Relação N:M entre Emprestimo ↔ Livro (via EmprestimoLivro).

Script de Criação — schema_biblioteca.sql

-- Criar o banco

```
CREATE DATABASE biblioteca;
```

-- Conectar ao banco

```
\c biblioteca;
```

-- Tabela Autor

```
CREATE TABLE autor (
```

```
    id_autor SERIAL PRIMARY KEY,
```

```
    nome VARCHAR(100) NOT NULL
```

```
);
```

-- Tabela Livro

```
CREATE TABLE livro (
```

```
    id_livro SERIAL PRIMARY KEY,
```

```
    titulo VARCHAR(150) NOT NULL,
```

```
    ano_publicacao INT,
```

```
    id_autor INT REFERENCES autor(id_autor)
```

```
);
```

-- Tabela Aluno

```
CREATE TABLE aluno (
```

```
    id_aluno SERIAL PRIMARY KEY,
```

```
    nome VARCHAR(100) NOT NULL,
```

```
    curso VARCHAR(100) NOT NULL
```

```
);
```

Script de Inserts — dados_iniciais_biblioteca.sql

-- Autores

```
INSERT INTO autor (nome) VALUES  
('J. R. R. Tolkien'),  
('Machado de Assis'),  
('Clarice Lispector');
```

-- Livros

```
INSERT INTO livro (titulo, ano_publicacao,  
id_autor) VALUES  
('O Senhor dos Anéis', 1954, 1),  
('Dom Casmurro', 1899, 2),  
('A Hora da Estrela', 1977, 3),  
('O Hobbit', 1937, 1);
```

-- Alunos

```
INSERT INTO aluno (nome, curso) VALUES  
('Ana Souza', 'Sistemas de Informação'),  
('Bruno Silva', 'Engenharia de Software');
```

-- Empréstimos

```
INSERT INTO emprestimo (data_emprestimo,  
id_aluno) VALUES  
('2025-08-20', 1),  
('2025-08-21', 2);
```

Script de Inserts — dados_iniciais_biblioteca.sql

-- EmprestimoLivro (associativa)

INSERT INTO emprestimo_livro (id_emprestimo, id_livro) VALUES

(1, 1), -- Ana Souza pegou O Senhor dos Anéis

(1, 2), -- Ana Souza pegou Dom Casmurro

(2, 3); -- Bruno Silva pegou A Hora da Estrela

BANCO DE DADOS RELACIONAL

Subconsultas (subqueries)

Objetivos da aula



- ✓ Tipos de subconsultas: scalar, coluna, tabela e correlacionada.
- ✓ Subconsultas em **SELECT**, **FROM** e **WHERE**.

Conceitos rápidos

- ❑ **Subconsulta (subquery):** uma consulta SQL dentro de outra.
 - ❑ Pode retornar: um único valor (scalar), uma coluna, ou uma tabela inteira (linha(s)).
- ❑ **Subconsulta correlacionada:** refere-se a colunas da query externa; reavaliada para cada linha da query externa.

Subconsultas em SELECT (scalar subquery)

- ❑ **Contexto (Biblioteca):** quero listar cada autor com a média de páginas dos seus livros (subconsulta no SELECT).
- ❑ **Esquema usado:**
- ❑ *autor(id_autor, nome)*
- ❑ *livro(id_livro, titulo, num_paginas, id_autor)*

SQL com subquery scalar:

```
SELECT a.nome,
       (SELECT AVG(l2.num_paginas)
        FROM livro l2
        WHERE l2.id_autor = a.id_autor) AS
       media_paginas
FROM autor a;
```

Explicação:

Para cada autor ***a***, a subconsulta calcula a média de ***num_paginas*** dos livros desse autor.

A subquery é correlacionada (***l2.id_autor = a.id_autor***).

Subconsultas em SELECT (scalar subquery)

- ❑ Você tem duas tabelas no **BD Biblioteca**:
 - ❑ Tabela Autor e tabela Livros:

id_autor	nome
1	Machado de Assis
2	Clarice Lispector
3	Graciliano Ramos

id_livro	titulo	num_paginas
1	Dom Casmurro	250
2	Memórias Póstumas	200
3	A Hora da Estrela	120
4	Vidas Secas	180
5	São Bernardo	200

Subconsultas em SELECT (scalar subquery)

- ❑ **O problema que queremos resolver:**
 - ❑ Queremos montar **uma lista de autores** e, ao lado de cada autor, mostrar **a média de páginas dos seus livros**.
 - ❑ Ou seja, o resultado final precisa ter **uma linha por autor**, mas precisamos **calcular algo (a média)** olhando **para outra tabela (livro)**.

- ❑ **Como o banco “pensa”:**
 - ❑ O PostgreSQL **lê linha por linha da tabela externa (autor)**, e para **cada linha**, ele executa uma **subconsulta** (uma mini-query dentro da query principal).

Subconsultas em SELECT (scalar subquery)

❑ O código SQL completo:

SELECT

a.nome,

(SELECT AVG(l2.num_paginas)

FROM livro l2

WHERE l2.id_autor = a.id_autor) AS media_paginas

FROM autor a;

Subconsultas em SELECT (scalar subquery)

- ❑ **Explicação passo a passo:**

- 1-Parte externa:** “a tabela principal”

- SELECT a.nome*

- FROM autor a;*

- ❑ Aqui o banco vai percorrer a tabela **autor**, linha por linha:

- ❑ 1ª linha: Machado de Assis

- ❑ 2ª linha: Clarice Lispector

- ❑ 3ª linha: Graciliano Ramos

- ❑ Até aqui, nada demais.

Subconsultas em SELECT (scalar subquery)

- ❑ **Explicação passo a passo:**

- 2- Parte interna: "a mini-query"**

(SELECT AVG(l2.num_paginas)

FROM livro l2

WHERE l2.id_autor = a.id_autor)

- ❑ Essa parte **roda uma consulta dentro da consulta principal**.

O truque é essa condição:

WHERE l2.id_autor = a.id_autor

- ❑ Ela faz o banco comparar o **id_autor da tabela externa (autor a)** com o **id_autor da tabela interna (livro l2)**.
 - ❑ Ou seja, **para cada autor**, ele procura **todos os livros daquele autor**, calcula a média de páginas e devolve **um número** (um valor escalar).

Subconsultas em SELECT (scalar subquery)

❑ Explicação passo a passo:

3- Juntando tudo

O banco executa assim internamente:

Autor (a.nome)	Subquery executada	Resultado da subquery
Machado de Assis	AVG (num_paginas) dos livros com id_autor=1	225
Clarice Lispector	AVG (num_paginas) dos livros com id_autor=2	120
Graciliano Ramos	AVG (num_paginas) dos livros com id_autor=3	190

Subconsultas em SELECT (scalar subquery)

- ❑ Explicação passo a passo:
4- Resultado final

Autor	Média de Páginas
Machado de Assis	225
Clarice Lispector	120
Graciliano Ramos	190

Imaginem que o banco está com uma lupa na mão.

Ele olha o autor Machado de Assis e pensa:
'Quais livros são dele?'

Vai até a tabela de **livros**, pega os dois, calcula a **média das páginas** (225), anota do lado do nome dele, e passa para o próximo autor.

Ele repete isso para cada autor.

Essa repetição é a **subconsulta** correlacionada em ação.

Subconsultas em SELECT (scalar subquery)

- ❑ Explicação passo a passo:
 4- Resultado final

Autor	Média de Páginas
Machado de Assis	225
Clarice Lispector	120
Graciliano Ramos	190

Conceito importante

- Chamamos esse tipo de subconsulta de **subconsulta correlacionada** porque ela **depende** de um valor da **query externa** (a.id_autor).

- É uma subconsulta escalar porque **devolve apenas um valor** (um número, não uma lista ou tabela).

Subconsultas em SELECT (scalar subquery)

❑ Variações:

1-Contar livros por autor:

```
SELECT a.nome,  
       (SELECT COUNT(*) FROM livro l2 WHERE l2.id_autor = a.id_autor) AS total_livros  
FROM autor a;
```

2-Maior número de páginas por autor:

```
SELECT a.nome,  
       (SELECT MAX(l2.num_paginas) FROM livro l2 WHERE l2.id_autor = a.id_autor) AS  
maior_livro  
FROM autor a;
```

- ❑ Essas variações ajudam a perceber que a estrutura é a mesma, só muda a **função agregadora**.

Subconsulta em SELECT (Tema: limnologia_db)

- ❑ O banco de dados **limnologia_db** armazena informações sobre campanhas, reservatórios, parâmetros e medições limnológicas (como temperatura, pH, turbidez etc.).
- ❑ Você quer criar um **relatório** que mostre **a média de valores medidos (campo valor)** de um determinado parâmetro em **cada reservatório**.
- ❑ Em outras palavras:
"Qual é a média dos valores medidos no reservatório A, no reservatório B, no reservatório C...?"

Subconsulta em SELECT (Tema: limnologia_db)

- ❑ Estrutura simplificada das tabelas:

Tabela reservatorio

id_reservatorio	nome
1	Jaguari
2	Paraibuna
3	Cachoeira do França

Tabela parametro

id_parametro	nome_parametro
1	pH

Tabela serie_temporal

id_serie	id_reservatorio
1	1
2	1
3	2
4	3
5	3

Subconsulta em SELECT (Tema: limnologia_db)

- ❑ **O problema que queremos resolver:**
 - ❑ O INPE quer visualizar, em um relatório simples:
 - ❑ "Qual é a **média do valor do pH** em cada reservatório?"

Subconsulta em SELECT (Tema: limnologia_db)

- ❑ **Como o PostgreSQL pensa:**

- ❑ **Assim como no exemplo da Biblioteca, o banco vai:**

- 1-Percorrer cada linha da tabela **reservatorio** (um por um);

- 2-Para cada reservatório, executar uma **subconsulta** que calcula a média dos valores (AVG(valor)) na tabela **serie_temporal**, onde o id_reservatorio seja o mesmo da linha atual.

Subconsulta em SELECT (Tema: limnologia_db)

❑ Código SQL:

SELECT

r.nome AS reservatorio,

(SELECT AVG(s.valor)

FROM serie_temporal s

INNER JOIN parametro p ON s.id_parametro = p.id_parametro

WHERE s.id_reservatorio = r.id_reservatorio

AND p.nome_parametro = 'pH'

) AS media_ph

FROM reservatorio r;

Subconsulta em SELECT (Tema: limnologia_db)

- ❑ **Explicação passo a passo:**
 - ❑ **1-Parte externa (a tabela principal)**
SELECT r.nome FROM reservatorio r;
- ❑ **O PostgreSQL vai percorrer cada reservatório:**
 - Jaguari
 - Paraibuna
 - Cachoeira do França

Subconsulta em SELECT (Tema: limnologia_db)

- ❑ Explicação passo a passo:

- ❑ Parte interna (a mini-consulta)

(SELECT AVG(s.valor)

FROM serie_temporal s

INNER JOIN parametro p ON s.id_parametro = p.id_parametro

WHERE s.id_reservatorio = r.id_reservatorio

AND p.nome_parametro = 'pH')

- ❑ Aqui é onde a **mágica** acontece.
- ❑ Para cada reservatório (linha atual da tabela externa), o banco:
 - ❑ Vai até a tabela **serie_temporal**,
 - ❑ Filtra os registros do mesmo **id_reservatorio**,
 - ❑ Filtra apenas as medições do parâmetro **pH**,
 - ❑ Calcula a **média dos valores**.

Subconsulta em SELECT (Tema: limnologia_db)

- ❑ Explicação passo a passo:
- ❑ 3-O que o banco realmente faz (em sequência)

Reservatório (r.nome)	Subconsulta executada	Resultado da subquery
Jaguari	média dos valores em serie_temporal onde id_reservatorio = 1	7.2
Paraibuna	média dos valores onde id_reservatorio = 2	6.9
Cachoeira do França	média dos valores onde id_reservatorio = 3	7.6

Subconsulta em SELECT (Tema: limnologia_db)

- ❑ Explicação passo a passo:
- ❑ 4-Resultado final

Reservatório	Média pH
Jaguari	7.2
Paraibuna	6.9
Cachoeira do França	7.6

- ❑ Vamos usar a metáfora da **planilha inteligente**:
- ❑ Imaginem que o PostgreSQL tem uma planilha com os nomes dos reservatórios.
 - ❑ Para cada linha, ele abre outra planilha (a tabela **serie_temporal**) e faz o cálculo da **média** do **pH** apenas para aquele reservatório.
 - ❑ Depois **anota** o resultado do lado do nome do reservatório e **passa para o próximo**."

Subconsulta em SELECT (Tema: limnologia_db)

- ❑ **Conceitos que é importante fixar:**
 - ❑ É uma **subconsulta correlacionada**, pois usa um campo da query externa (**r.id_reservatorio**) dentro da subquery.
 - ❑ É **escalar (scalar subquery)**, porque retorna **apenas um valor** (um número).
 - ❑ É útil quando queremos calcular algo **para cada linha da tabela principal**, sem precisar de **JOIN + GROUP BY**.

Subconsulta em SELECT (Tema: limnologia_db)

Variações para demonstrar:

1-Contar o número de medições por reservatório

```
SELECT r.nome,  
       (SELECT COUNT(*) FROM serie_temporal s WHERE s.id_reservatorio =  
        r.id_reservatorio) AS total_medicoes  
FROM reservatorio r;
```

Subconsulta em SELECT (Tema: limnologia_db)

Variações para demonstrar:

2-Encontrar o maior valor de pH por reservatório

```
SELECT r.nome,  
       (SELECT MAX(s.valor)  
        FROM serie_temporal s  
        INNER JOIN parametro p ON s.id_parametro = p.id_parametro  
        WHERE s.id_reservatorio = r.id_reservatorio  
        AND p.nome_parametro = 'pH'  
       ) AS ph_maximo  
FROM reservatorio r;
```


Subconsulta em SELECT (Tema: limnologia_db)

Dica:

- ❑ **Agora você vê que o SQL funciona como uma conversa entre duas tabelas:**

A tabela **reservatorio** pergunta à **serie_temporal**:

- *'Quais foram as minhas medições?'*

E a **serie_temporal** responde:

- *'A sua média foi 7.2!'*

- ❑ Essa conversa acontece linha a linha, e por isso chamamos de **subconsulta correlacionada**.

Exercício Prático (Individual)

Subconsultas no SELECT (limnologia_db)

- ❑ **Objetivo**
- ❑ Compreender **como funcionam as subconsultas escalares (scalar subqueries)** no PostgreSQL, usando o contexto real do projeto **limnologia_db**.
Ao final, você será capaz de:
 - ❑ Criar uma **subconsulta** dentro do **SELECT**;
 - ❑ Entender a diferença entre a **consulta externa** e a **subconsulta interna**;
 - ❑ Explicar o conceito de **subconsulta correlacionada**.

Exercício Prático (Individual)

Subconsultas no SELECT (limnologia_db)

❑ Cenário: Dados de pH nos reservatórios

O INPE deseja gerar um relatório que mostre **a média, o maior e o menor valor de pH** medido em cada reservatório.

Essas informações estão armazenadas em três tabelas:

Tabela	Campos principais	Descrição
reservatorio	id_reservatorio, nome	Lista dos reservatórios monitorados
serie_temporal	id_serie, id_reservatorio, id_parametro, valor, data_hora	Valores registrados por parâmetro
parametro	id_parametro, nome_parametro	Tipos de parâmetros coletados (pH, temperatura, turbidez etc.)

Exercício Prático (Individual)

Subconsultas no SELECT (limnologia_db)

Passo 1: Conferir os dados existentes

- ❑ Antes de tudo, verifique se o banco contém registros.
- ❑ Garanta que há dados com o parâmetro '*pH*' e valores medidos.

Passo 2: Criar a consulta base

- ❑ Comece simples, mostrando apenas os nomes dos reservatórios.
- ❑ Essa é a tabela **externa**, o ponto de partida.
Queremos adicionar informações calculadas ao lado de cada reservatório.

Passo 3: Adicionar a subconsulta para calcular a média do pH

- ❑ A subconsulta é executada **uma vez para cada reservatório**.
- ❑ Ela pega apenas os registros da tabela *serie_temporal* que correspondem àquele *reservatorio* e ao parâmetro pH.
- ❑ Retorna **um único valor numérico**, que é mostrado na coluna *media_ph*.

Passo 4: Adicionar mais duas subconsultas (mínimo e máximo)

- ❑ Com isso, o PostgreSQL vai executar **três subconsultas diferentes** para cada reservatório: uma calcula a média, outra o menor valor, e outra o maior valor de pH

Exercício Prático (Individual)

Subconsultas no SELECT (limnologia_db)

Passo 5: Verificando o resultado:

- ❑ O resultado esperado será algo assim:

Reservatório	Média pH	pH Mínimo	pH Máximo
Jaguari	7.2	7.1	7.3
Paraibuna	6.9	6.9	6.9
Cachoeira do França	7.6	7.5	7.7

- ❑ **Observem:**

- ❑ Como cada subconsulta usa o mesmo filtro (*WHERE s.id_reservatorio = r.id_reservatorio*)?
- ❑ O que aconteceria se retirássemos esse filtro?
- ❑ O que significa o termo “**correlacionada**”?
- ❑ A subconsulta “**depende**” da linha atual da consulta externa, por isso chamamos de correlacionada.

Exercício Prático (Individual)

Subconsultas no SELECT (limnologia_db)

Dica:

- ❑ O PostgreSQL está fazendo o trabalho de um analista que percorre a lista de reservatórios e pergunta:
 - *'Qual é a média do pH do Jaguari?'*
 - *'E do Paraibuna?'*
 - *'E do Cachoeira do França?'*
- ❑ **Para cada pergunta, ele abre a tabela de medições, calcula e anota o resultado.**

O que veremos na próxima aula

- ❑ Consolidação de consultas com **subconsultas + GROUP BY**.
- ❑ Como combinar **subqueries** com agregações.
- ❑ Preparação final para o **Requisito BDR.02 — Funções Agrupadoras**.



Referências Bibliográfica da Aula

Livros:

Elmasri & Navathe (2010). Sistemas de Banco de Dados.

Silberschatz et al. (2011). Sistemas de Banco de Dados.

Links úteis:

 [PostgreSQL Docs](#)

 [DBDiagram.io](#)

Bibliografia Básica

- ❑ DATE, C. J. **Introdução a sistemas de bancos de dados**. Rio de Janeiro, Elsevier: Campus, 2004.
- ❑ ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**. 7 ed. São Paulo: Pearson, 2018.
- ❑ SILBERSCHATZ, A.; SUNDARSHAN, S.; KORTH, H. F. **Sistema de banco de dados**. Rio de Janeiro: Elsevier Brasil, 2016.

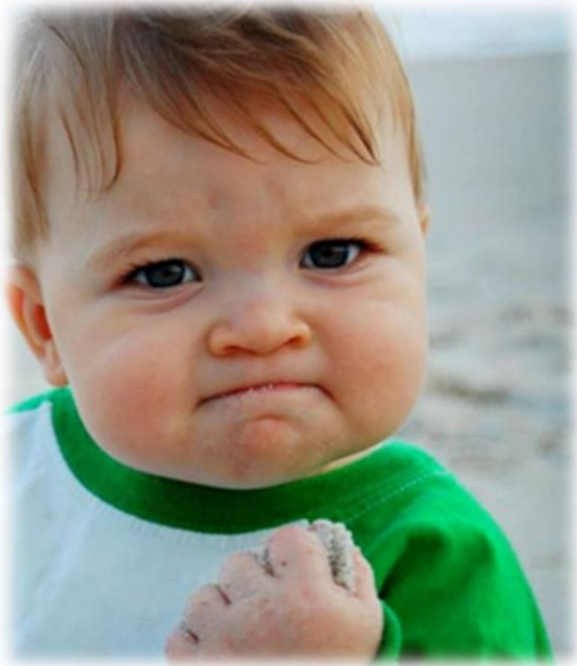
Bibliografia Complementar

- ❑ BEAULIEU, A. **Aprendendo SQL**. São Paulo: Novatec, 2010.
- ❑ GILLENSON, M. L. **Fundamentos de Sistemas de Gerência de Banco de Dados**. Rio de Janeiro: LTC, 2006.
- ❑ MACHADO, F. N. R. **Banco de Dados: Projeto e Implementação**. São Paulo: Érica, 2005.
- ❑ OTEY, M; OTEY, D. **Microsoft SQL Server 2005: Guia do Desenvolvedor**. Rio de Janeiro: Ciência Moderna, 2007.
- ❑ RAMAKRISHNAN, R.; GEHRKE, J. **Sistemas de Gerenciamento de Bancos de Dados**. 3 ed. Porto Alegre: Bookman, 2008.
- ❑ ROB, P; CORONEL, C. **Sistemas de Banco de Dados: Projeto, Implementação e Gerenciamento**. 8 ed. São Paulo: Cengage Learning, 2011.
- ❑ TEOREY, T; LIGHTSTONE, S; NADEAU, T. **Projeto e Modelagem de Bancos de Dados**. São Paulo: Campus, 2006.

Dúvidas?



Considerações Finais



**Professora:
Lucineide Pimenta**

Bom descanso à todos!

